Binary Tree — Pattern Recognition & Universal Notes

These notes are meant for quick revision before solving any binary tree problem. They focus on recognizing traversal patterns, recursion structures, and common design templates used across most problems.

---

1. How to Recognize a Binary Tree Problem

Typical signals:

- "Traverse the tree"
- "Find depth / height"
- "Lowest common ancestor"
- "Path from root"
- "Validate tree property"
- "Level order"

Binary tree problems are usually about **tree traversal + local computation**.

Key mindset:

Every node is the **root of its own subtree**.

Solve the problem for one node assuming it works for its children.

---

2. Binary Tree Node Structure

```
struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;

    TreeNode(int x) {
        val = x;
        left = NULL;
        right = NULL;
    }
};
```

Each node contains:

- value
- left child
- right child

---

3. The Core Recursive Idea

Most tree problems follow this pattern:

1. Solve for left subtree
2. Solve for right subtree
3. Combine results

Generic recursive template:

```
ReturnType dfs(TreeNode* root) {

    if (root == NULL)
        return base_value;

    auto left = dfs(root->left);
    auto right = dfs(root->right);

    return combine(left, right, root);
}
```

This single idea solves a huge number of tree problems.

---

4. Three Depth First Traversals

Preorder

```
Root → Left → Right
```

Template:

```
void preorder(TreeNode* root) {
    if (!root) return;
```

```
    visit(root);
    preorder(root->left);
    preorder(root->right);
}
```

Used when processing root before children.

Inorder

```
Left → Root → Right
```

Template:

```
void inorder(TreeNode* root) {
    if (!root) return;

    inorder(root->left);
    visit(root);
    inorder(root->right);
}
```

Important property:

For **Binary Search Trees**, inorder traversal gives **sorted order**.

Postorder

```
Left → Right → Root
```

Template:

```
void postorder(TreeNode* root) {
    if (!root) return;

    postorder(root->left);
    postorder(root->right);
    visit(root);
}
```

Used when decisions depend on children.

Example:

- height
- diameter
- balanced tree

---

5. Breadth First Search (Level Order)

Uses a queue.

Template:

```
queue<TreeNode*> q;
q.push(root);

while (!q.empty()) {

    TreeNode* node = q.front();
    q.pop();

    if (node->left) q.push(node->left);
    if (node->right) q.push(node->right);
}
```

Used for:

- level order traversal
- shortest distance in tree
- zigzag traversal

---

6. Height / Depth Pattern

Very common tree problem.

Template:

```
int height(TreeNode* root) {

    if (!root)
```

```
        return 0;

    return 1 + max(height(root->left), height(root->right));
}
```

Time complexity:

```
O(n)
```

---

7. Path Problems Pattern

Used when problem mentions **path from root**.

Typical template:

```
void dfs(TreeNode* root, vector<int>& path) {

    if (!root) return;

    path.push_back(root->val);

    if (leaf node)
        save path;

    dfs(root->left, path);
    dfs(root->right, path);

    path.pop_back();
}
```

This pattern appears in:

- path sum
- root to leaf paths

---

8. Lowest Common Ancestor Pattern

Classic recursive idea.

Template:

```
TreeNode* lca(TreeNode* root, TreeNode* p, TreeNode* q) {

    if (!root || root == p || root == q)
        return root;

    TreeNode* left = lca(root->left, p, q);
    TreeNode* right = lca(root->right, p, q);

    if (left && right)
        return root;

    return left ? left : right;
}
```

---

9. Diameter / Global Variable Pattern

Some problems need a **global answer** updated during DFS.

Template:

```
int ans = 0;

int dfs(TreeNode* root) {

    if (!root)
        return 0;

    int left = dfs(root->left);
    int right = dfs(root->right);

    ans = max(ans, left + right);

    return 1 + max(left, right);
}
```

Used in:

- diameter of tree
- max path sum

---

10. Mental Checklist Before Coding

Ask yourself:

1. Is this a traversal problem?
2. Which traversal fits best?
3. Do I need DFS or BFS?
4. What should the recursive function return?
5. Do I need global state?

Once these are clear, implementation becomes straightforward.

---

11. Pattern Recognition Summary

Most binary tree problems fall into these categories:

Pattern	Example
DFS traversal	Preorder / Inorder / Postorder
BFS traversal	Level order
Height / depth	Max depth
Path problems	Path sum
LCA	Lowest common ancestor
Global answer	Diameter

Recognizing the pattern is the key step.

---

12. Universal Binary Tree Templates

DFS recursion

```
dfs(node)
```

Height pattern

```
1 + max(left, right)
```

BFS queue

```
queue
```

Path tracking

```
push → recurse → pop
```

These templates solve the majority of binary tree problems.